

Bases de Datos para Inteligencia Artificial

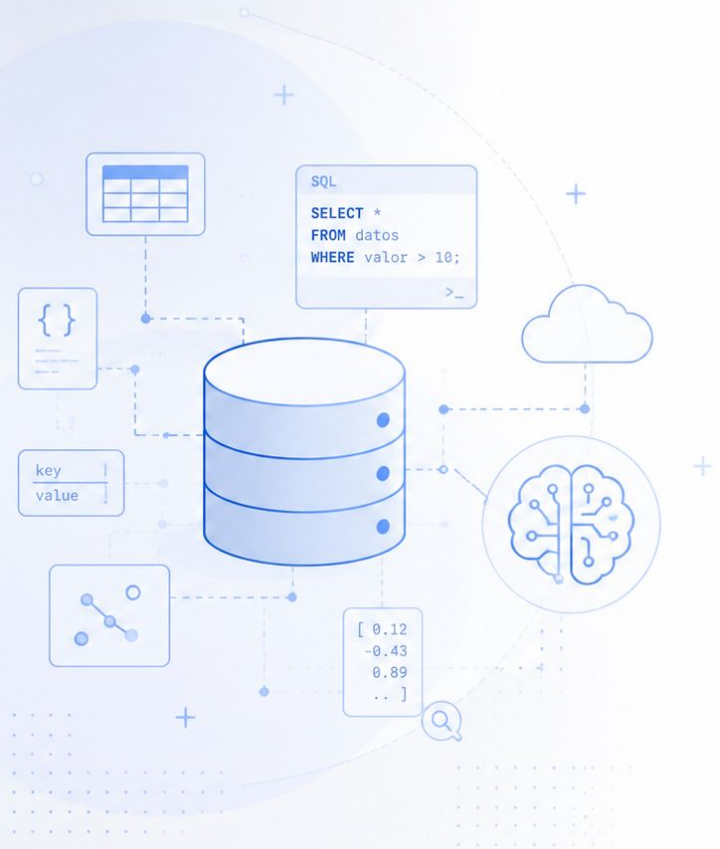
Especialización en Inteligencia Artificial

Docente:

Lacheski, Martín Anibal

Clase 8:

Proyecto Integrador: de los conceptos a un sistema verificable



Objetivo de la clase

Integrar los conceptos vistos durante la materia

Al finalizar la clase, deberíamos poder:

- reconocer cómo se conectan las decisiones de modelado, almacenamiento y consulta;
- identificar qué tecnología resuelve cada necesidad;
- comprender cómo se integran datos relacionales, documentos y vectores;
- observar cómo se aplica el aislamiento de datos;
- relacionar el caso práctico con los contenidos de las clases anteriores.

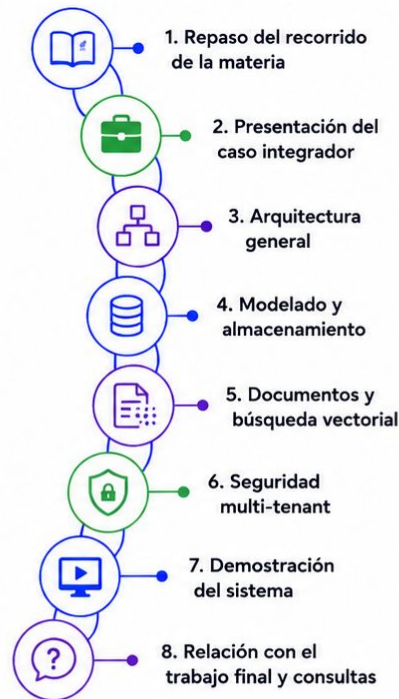


El objetivo no es incorporar nueva teoría,
sino observar cómo se articulan las decisiones estudiadas.

Bases de Datos para IA

Agenda del día

- Repaso del recorrido de la materia.
- Presentación del caso integrador.
- Arquitectura general.
- Modelado y almacenamiento.
- Documentos y búsqueda vectorial.
- Seguridad multi-tenant.
- Demostración del sistema.
- Relación con el trabajo final.
- Consultas.



Hoy vamos a recorrer el caso integrador paso a paso, vinculándolo con los contenidos trabajados durante la materia.

¿Qué fuimos construyendo?

De los datos individuales a una arquitectura completa

Durante la materia analizamos:

- bases de datos y tipos de información;
- modelado y SQL;
- diseño relacional;
- alternativas NoSQL;
- arquitecturas de datos;
- búsqueda vectorial;
- transacciones y escalabilidad;
- seguridad y aislamiento.

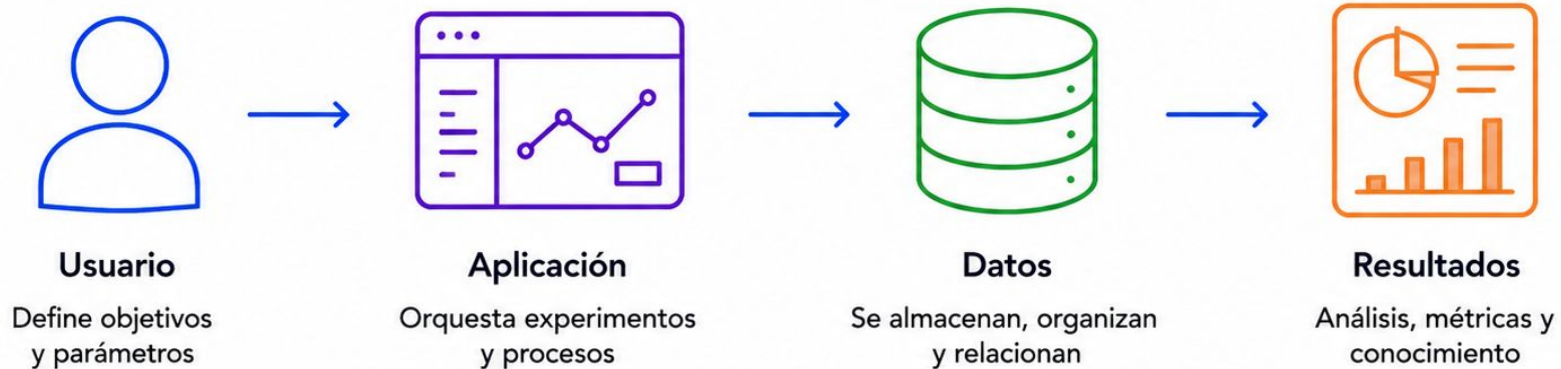


Cada tema resolvió una parte diferente del problema de gestionar datos.

Caso práctico integrador

Una misma solución para conectar los conceptos

Vamos a recorrer un sistema de experimentación para proyectos de Inteligencia Artificial.



El caso integrador permite conectar decisiones de modelado, almacenamiento, consulta, seguridad y escalabilidad.

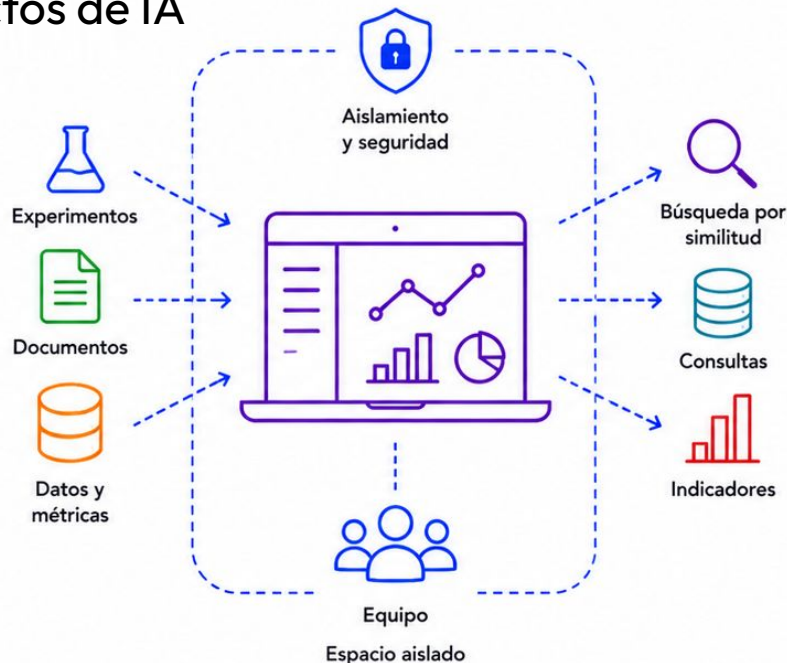
¿Qué problema se quiere resolver?

Un espacio de experimentación para proyectos de IA

Cada equipo puede:

- registrar experimentos;
- almacenar resultados y métricas;
- cargar documentos;
- recuperar información por similitud;
- consultar sus datos;
- visualizar indicadores.

Cada equipo trabaja dentro de su propio espacio.



El sistema necesita almacenar diferentes tipos de datos sin perder integridad ni aislamiento.

¿Qué vamos a hacer?

Seguir el dato de punta a punta



La demostración sigue el recorrido del dato dentro del sistema.

¿Cómo está compuesta la solución?

Cada componente tiene una responsabilidad



Aplicación web

Interacción con las personas.



API

Operaciones, validaciones y reglas de acceso.



PostgreSQL

Datos relacionales, relaciones y seguridad.



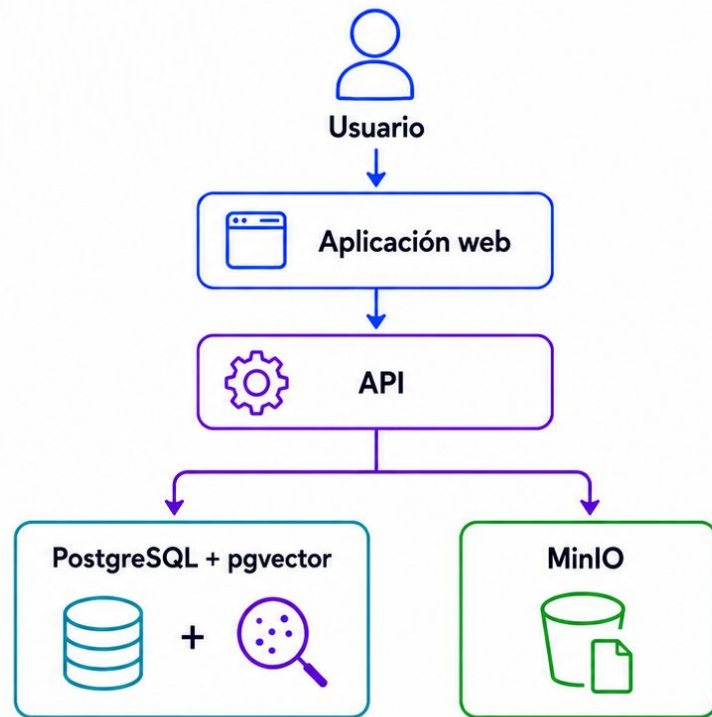
pgvector

Embeddings y búsqueda por similitud.



MinIO

Archivos originales.



La arquitectura separa responsabilidades según el tipo de dato y operación.

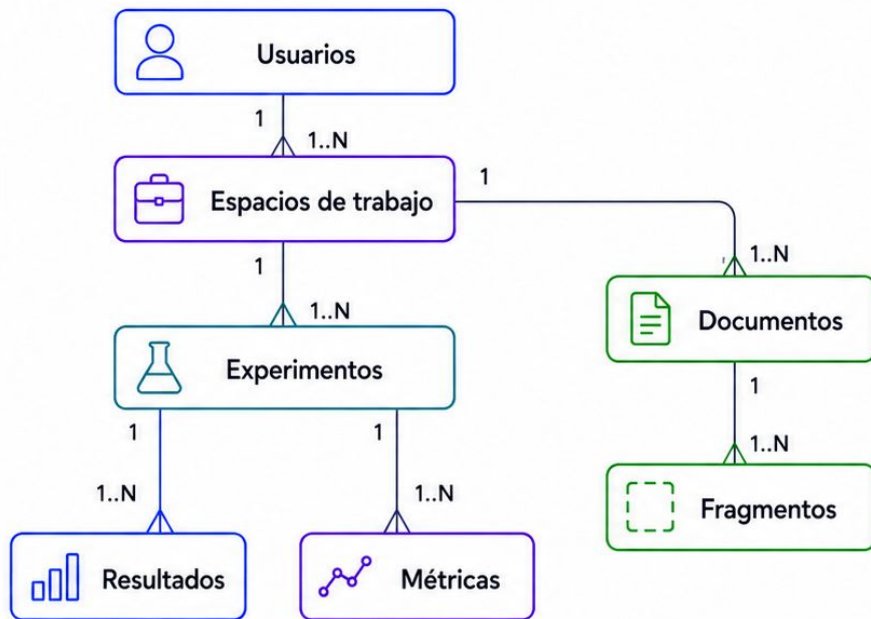
La base relacional

Representar las entidades principales del sistema

El modelo relaciona:

- usuarios;
- espacios de trabajo;
- experimentos;
- resultados;
- métricas;
- documentos;
- fragmentos.

Las claves y restricciones permiten mantener la integridad entre estos elementos.



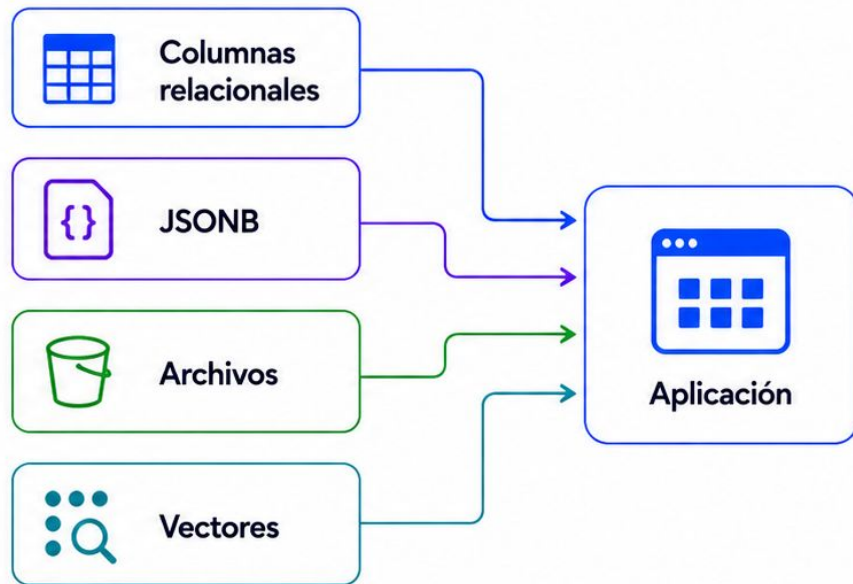
El modelo define qué información existe y cómo se relaciona.

Datos con distinta estructura

Elegir la representación según la necesidad

En una misma solución utilizamos:

- **columnas relacionales** para información estructurada;
- **JSONB** para información flexible;
- **archivos** para documentos originales;
- **vectores** para representar significado.



Una aplicación puede combinar diferentes representaciones sin necesitar una tecnología diferente para cada una.

Registrar evidencia

Separar el experimento de sus observaciones

Un experimento puede almacenar:

- descripción;
- estado;
- fechas;
- resultados;
- métricas asociadas.

Las métricas permiten representar distintos tipos de observaciones y comparar ejecuciones.



Separar entidades permite conservar mejor la historia y evitar información duplicada.

Incorporar datos no estructurados

Guardar el original y procesar su contenido

Cuando se carga un documento:

1. el archivo original se almacena;
2. se extrae su contenido;
3. el texto se divide en fragmentos;
4. cada fragmento conserva la referencia al documento.



El documento original y la información preparada para consulta cumplen funciones diferentes.

Representar significado

Preparar los fragmentos para búsqueda semántica

Cada fragmento obtiene una representación vectorial que permite compararlo con una consulta.



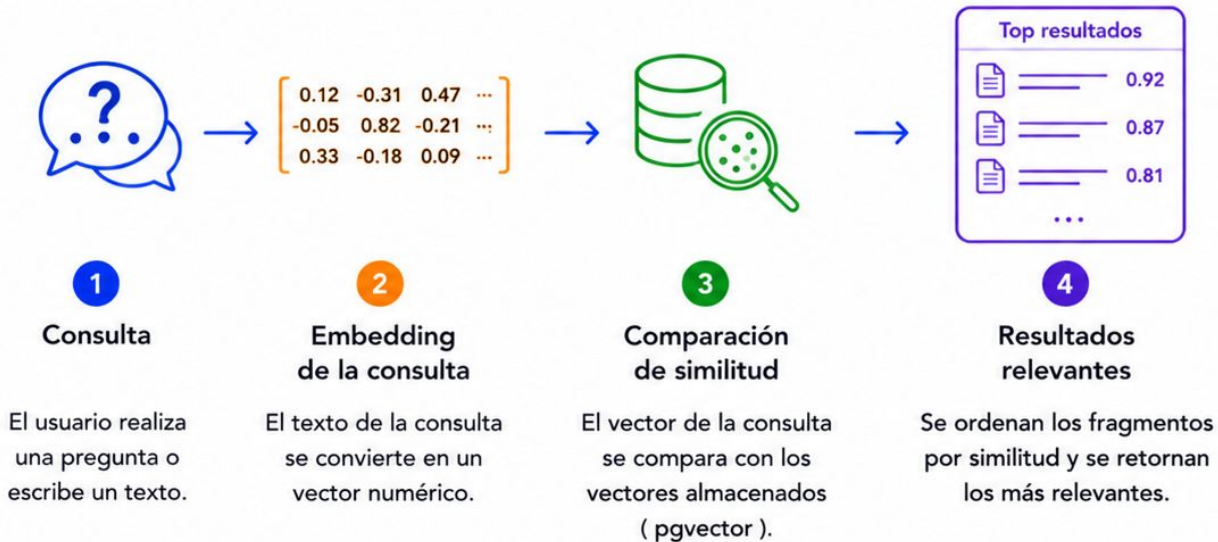
El embedding permite comparar significado;
el fragmento conserva la información que podremos recuperar.

Recuperar información

Encontrar los fragmentos más relacionados

Cuando se realiza una consulta:

1. se genera su embedding;
2. se compara con los vectores almacenados;
3. se ordenan los fragmentos por similitud;
4. se recuperan los resultados más relevantes.



La búsqueda vectorial permite recuperar información aunque no utilice exactamente las mismas palabras.

Consultar el sistema

Datos estructurados y evidencia documental

El sistema puede recuperar información desde:

Datos relacionales

- experimentos;
- resultados;
- métricas.

Documentos

- fragmentos recuperados por similitud.

Ambas fuentes pertenecen al mismo espacio de trabajo.



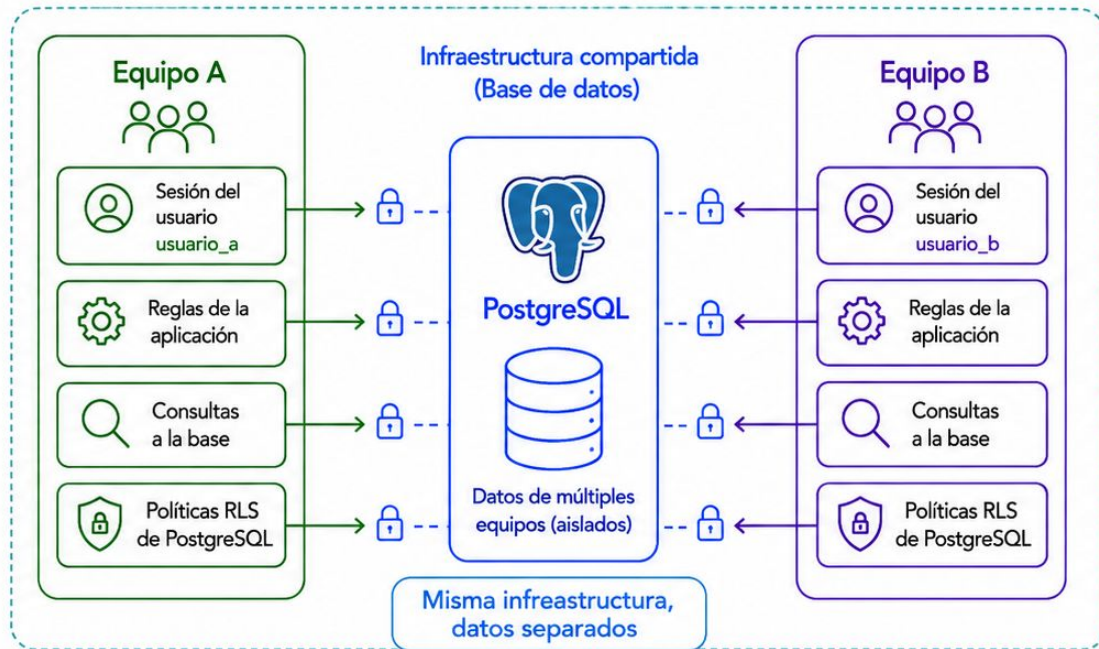
La respuesta puede combinar diferentes fuentes sin perder su procedencia.

Separar los espacios de trabajo

Cada equipo debe acceder únicamente a sus datos

El aislamiento se aplica en diferentes niveles:

- sesión del usuario;
- reglas de la aplicación;
- consultas a la base;
- políticas RLS de PostgreSQL.



Compartir infraestructura no significa compartir información.

¿Proteger en más de una capa?

No depender de una única validación



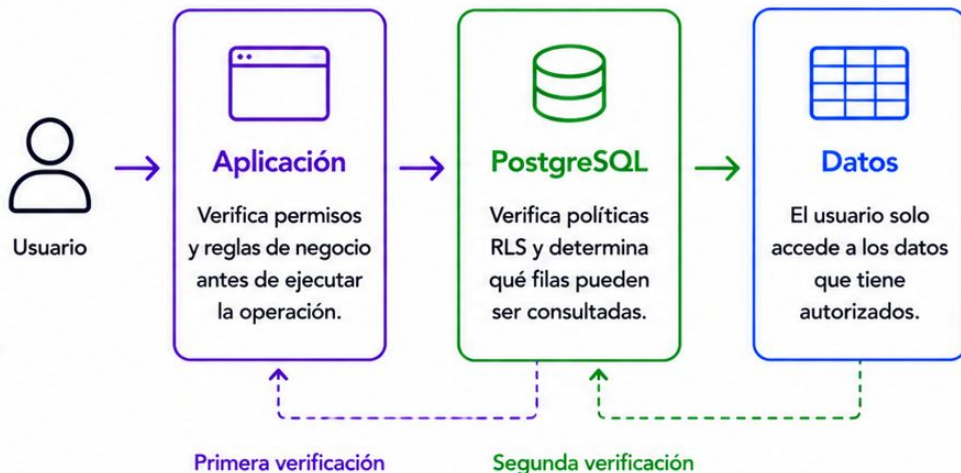
La **aplicación** verifica qué puede hacer cada usuario.



PostgreSQL vuelve a verificar qué filas puede consultar.



Esto permite que una consulta incorrecta no implique necesariamente una exposición de información.



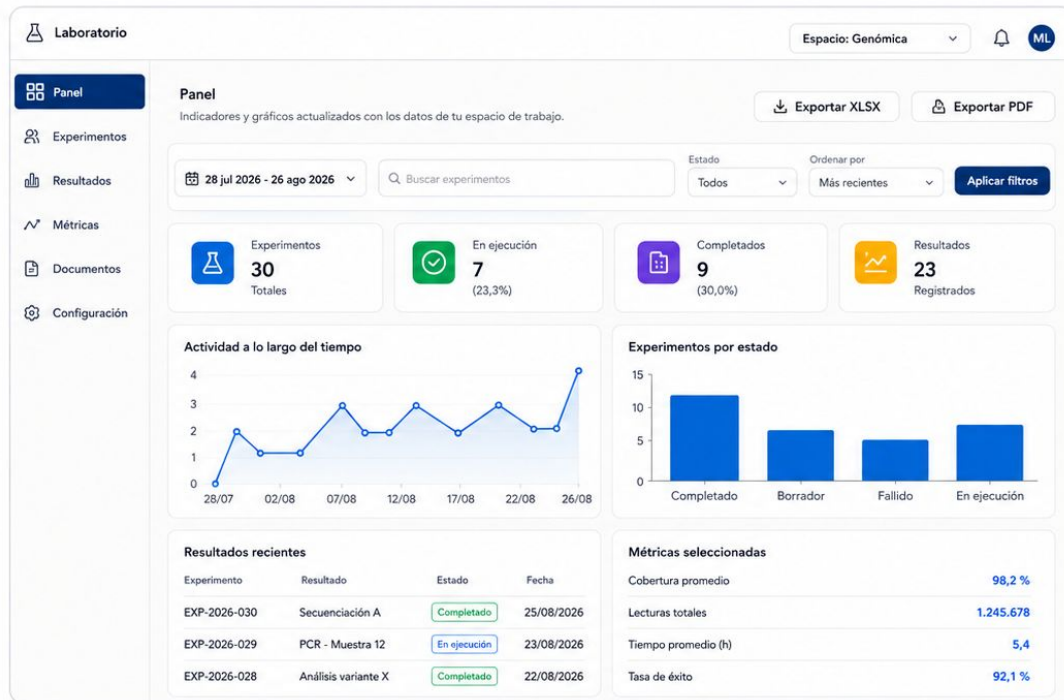
La seguridad no depende solamente de escribir correctamente cada consulta. Esta sintetiza buena parte de las actuales filmas sobre sesión, autorización, contexto y RLS sin entrar en los detalles de implementación.

Visualizar lo almacenado

Construir indicadores a partir de los datos

El panel puede mostrar:

- cantidad de experimentos;
- estados;
- resultados registrados;
- actividad a lo largo del tiempo;
- métricas seleccionadas.



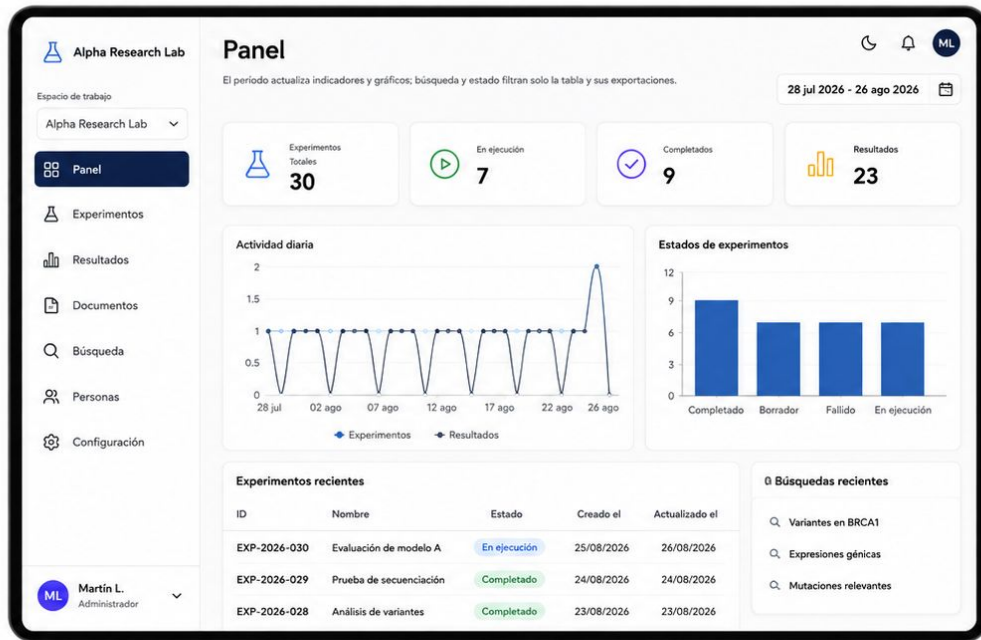
Los mismos datos utilizados para operar también pueden transformarse en información para analizar.

Veamos el sistema funcionando

Recorrer el caso de punta a punta

Vamos a:

- ingresar al sistema;
- crear un experimento;
- registrar un resultado;
- cargar un documento;
- realizar una búsqueda;
- consultar información;
- observar el panel.



Ahora vamos a observar en funcionamiento las decisiones que analizamos.

¿Se ven datos de otro espacio?

Intentar romper una de las garantías del sistema

Vamos a trabajar con:

A Tenant A

- experimentos;
- documentos;
- resultados propios.

B Tenant B

- información diferente.

Después intentaremos consultar desde A información perteneciente a B.



Resultado esperado

Los datos del otro espacio no deben aparecer.



Una buena forma de comprobar una política de aislamiento es intentar acceder a datos que no deberían estar disponibles.

Un mismo caso, distintos conceptos

Relacionar la práctica con las clases anteriores

Clase			Aplicación en el caso	
1		Tipos de datos y SGBD		Elección del SGBD y tipos (JSONB, vectores) según necesidades del caso.
2		Modelado y SQL		Diseño del esquema relacional y consultas SQL para el caso.
3		Integridad, relaciones, índices y JSONB		Reglas de integridad, relaciones, índices y uso de JSONB.
4		Criterios SQL y NoSQL		Elección de abordaje relacional y uso de JSONB según criterios.
5		Arquitectura y almacenamiento		Despliegue en la nube y estrategia de almacenamiento del caso.
6		Vectores y transacciones		Búsqueda semántica con vectores y transacciones para consistencia.
7		Multi-tenant, roles y RLS		Aislamiento por tenant con roles y RLS para proteger los datos.



El proyecto no agrega otra tecnología: articula las decisiones que fuimos estudiando.

¿Por qué estas tecnologías?

Elegir según el problema a resolver



PostgreSQL

Relaciones, integridad, consultas y seguridad.



pgvector

Búsqueda por similitud integrada con los datos.

MINIO

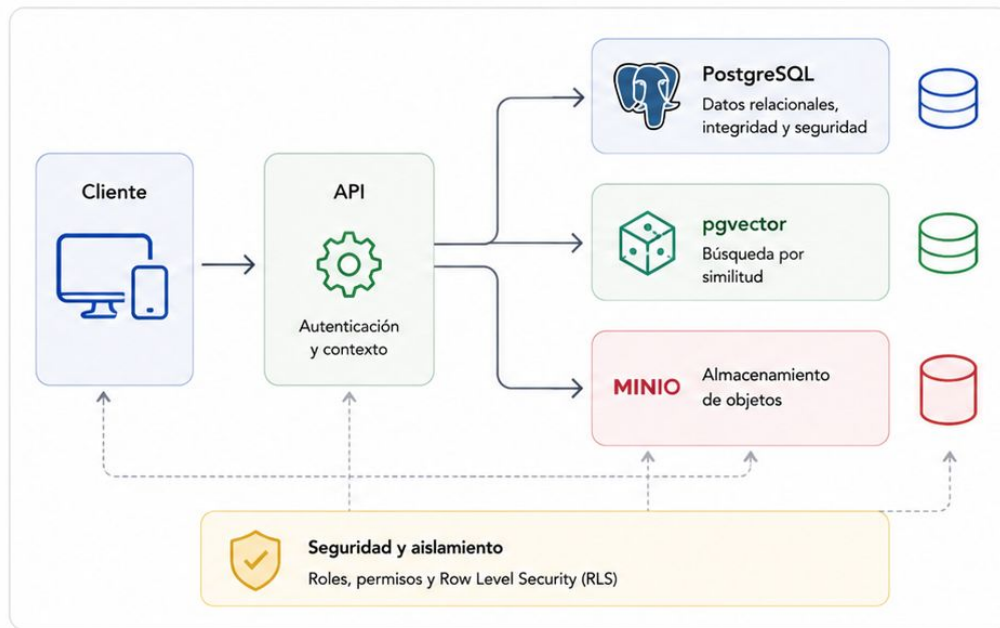
MinIO

Almacenamiento de objetos.



API

Control de operaciones y contexto.



La tecnología se selecciona a partir de los requerimientos, no al revés.

Alcance del caso

Una práctica académica, no una plataforma productiva

El caso no busca implementar:

- alta disponibilidad;
- procesamiento distribuido;
- grandes volúmenes reales;
- observabilidad completa;
- procesamiento asíncrono complejo;
- despliegue productivo.

El objetivo es poder observar las principales decisiones relacionadas con los datos.



Alta disponibilidad

No se implementa replicación ni failover.



Procesamiento distribuido

No se trabaja con sistemas distribuidos ni clústeres.



Grandes volúmenes reales

Se utilizan volúmenes de datos acotados y controlados.



Observabilidad completa

No se implementan métricas, logs y trazas avanzadas.



Procesamiento asíncrono complejo

No se incluyen colas, workers ni flujos event-driven.



Despliegue productivo

No se aborda la infraestructura ni el despliegue en producción.



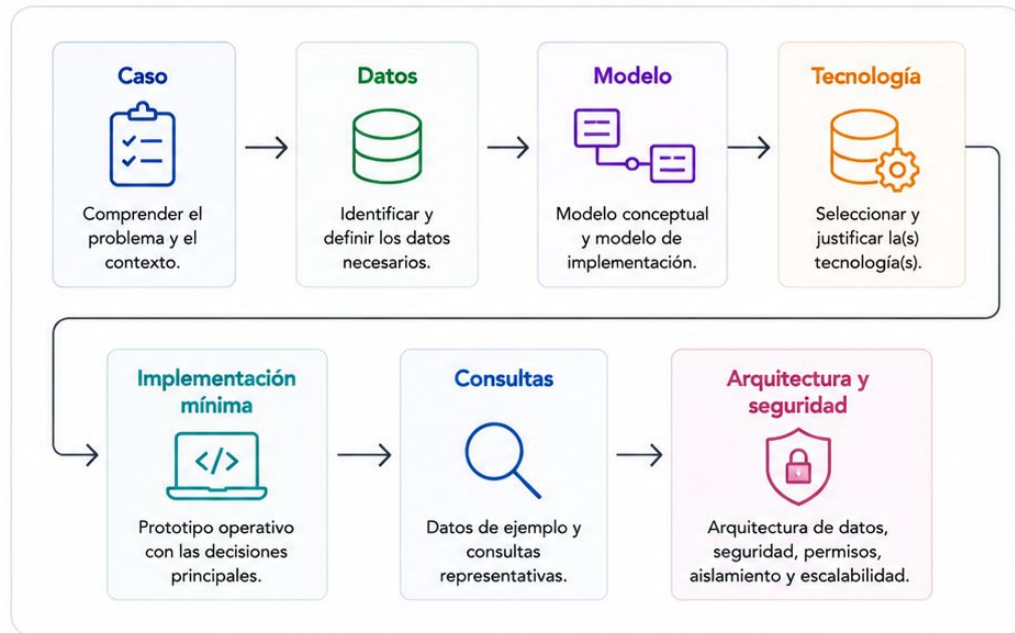
Definir los límites también forma parte del diseño de una solución.

¿Y el trabajo integrador?

Diseñar la solución de datos y demostrar sus decisiones principales

Para el caso elegido deberán:

- analizar el problema y los datos necesarios;
- construir el modelo conceptual y el modelo de implementación;
- justificar la tecnología o tecnologías seleccionadas;
- proponer la arquitectura de datos;
- definir seguridad, permisos y aislamiento;
- analizar escalabilidad y rendimiento;
- realizar una **implementación mínima**;
- incorporar datos de ejemplo y consultas representativas.



El caso desarrollado en clase utiliza PostgreSQL como propuesta guía.

En el TP, cada grupo deberá seleccionar y justificar la tecnología adecuada para su caso.

Antes de elegir una tecnología

Preguntas que deberían poder responder

- ¿Qué datos necesito almacenar?
- ¿Cómo se relacionan?
- ¿Cómo los voy a consultar?
- ¿Qué volumen y velocidad espero?
- ¿Necesito búsqueda por significado?
- ¿Quién puede acceder a cada dato?
- ¿Qué componentes podrían necesitar escalar?
- ¿Por qué esta arquitectura es adecuada?



Una buena solución comienza por las preguntas, no por las herramientas.



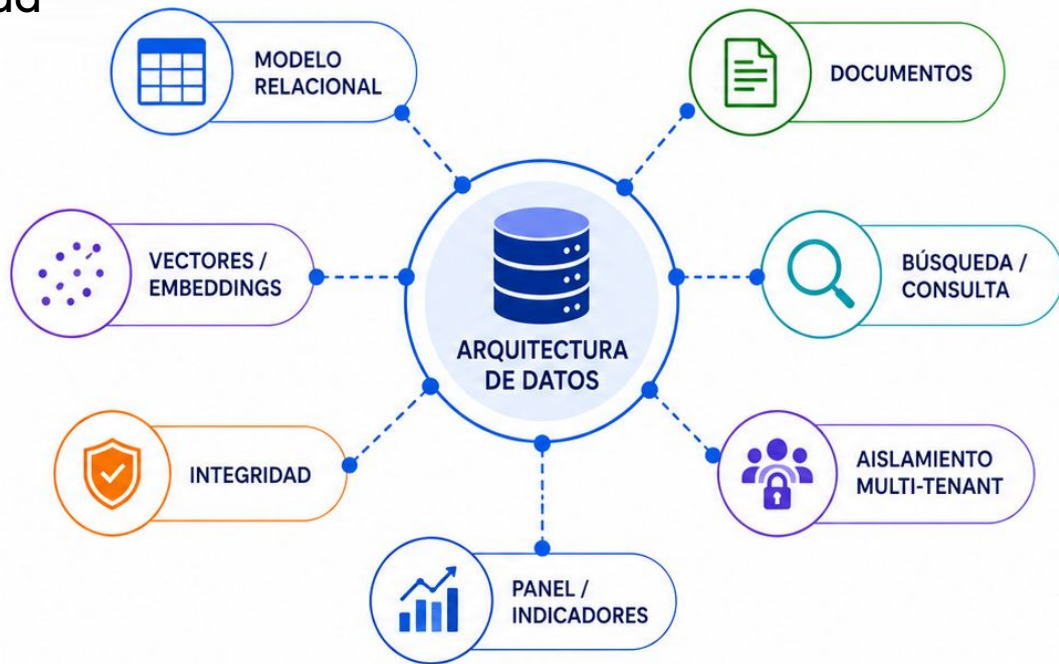
Veamos la práctica..

¿Qué observamos?

Una arquitectura de datos integrada

En el recorrido vimos cómo:

- modelar información estructurada;
- almacenar documentos;
- utilizar representaciones vectoriales;
- recuperar información;
- mantener integridad;
- aislar datos entre espacios;
- transformar datos en consultas e indicadores.



Una solución de IA necesita tanto buenos modelos como una arquitectura de datos capaz de sostenerlos.



Para ir cerrando..

Bases de Datos para IA

Elegir, diseñar y verificar

A lo largo de la materia recorrimos diferentes tecnologías y arquitecturas.

El objetivo no fue aprender una herramienta para cada problema, sino desarrollar criterios para:

- comprender los datos;
- modelarlos;
- almacenarlos;
- consultarlos;
- protegerlos;
- seleccionar la tecnología adecuada.



No existe una única base de datos para IA.

Existe una decisión de arquitectura adecuada para cada problema.

Consultas

Trabajo integrador y contenidos de la materia

Espacio para:

- dudas sobre el trabajo;
- alcance esperado;
- decisiones tecnológicas;
- arquitectura;
- modelado;
- seguridad;
- consultas generales de la materia.



Este espacio final es para resolver dudas y revisar decisiones del trabajo integrador.



Enlace a la encuesta



Muchas gracias